

2nd Assignment (1st part of big assignment)

Common requirements for the assignments

You need to create documentation (a pdf file) for the assignment. The program is only acceptable if you also have the documentation.

There are some things that are not defined, for example, exact numbers for durability loss, fuel consumption, etc. In these cases you are free to design and implement them as you see fit.

The documentation should include

- the description of the exercise (copied from here)
- use case diagram
- the class diagram (designed using Umbrello, draw.io, plantUML, etc.)

The code should include

- javadoc comments for each class and method (except getters and setters)
- unit tests for each of the important methods (not getters or setters)

1 University Library Management System

The university library management system maintains a diverse collection of items and facilitates borrowing and returning processes for members.

Items, have an id, a title, an author, page count, topic. The system tracks how many times an item has been borrowed. The item could be one a book, thesis, or research paper. Each item might have multiple copies and new copies can be added. If an item is borrowed, and there are more copies, it should still be available for others to be borrowed; if all copies were borrowed then it would not be available. When returning an item, it would be available for borrowing again if not damaged. A book could get damaged due to misuse and when returning it should be marked as damaged or not, if it's damaged the item needs repair and a repair must be requested.

An item can have a digital copy. By default the item wouldn't have one, but it can be created. When created, it would have same id as the item. Members can view the item, and if they want they can download the digital copy. The number of views and downloads is kept. The library should be able to find most viewed and most downloaded items.

To borrow items, users must be members of the library, they have their own id, name, email, and they have a category of student, faculty, or alumni. Each user keeps an active list of currently borrowed items and initially it should not exceed 3 per member. They should also keep the number of total borrowed items (when a book is successfully returned, then it is added to the list). For each 3 items that are returned, the member's loyalty would be appreciated by increasing the number of allowed items. However, to be able to consider a return of an item as a loyalty point, the condition of the item should be marked as preserved (not damaged).

The Library has a Repairs Section. It keeps count each time an item is sent to be repaired. If damage is severe (use randomness) then the book should be removed from the library to the archives of the library. If successfully repaired then it would normally be back to be available. The Library does a periodical check every now and then to send the items that need repair to repair Section, and to do the repair process.

The library stores the collection of items, It also tracks active transactions (borrowing and returning) and provides search functionalities by title and author to help users locate desired materials. The Library can add new items to its collection and manages the borrowing and returning of items. The library also keeps track of all their members, and can search for most active members. Also the people can find most popular items based on the number of times they have been borrowed: the most popular digital item,

and the most popular in general taking in account both physical and digital copies.

The main simulation must initialize the Library. Create items and add them to the library. Some items might have digital copies. Create members and make borrowing and returning transactions and other necessary operations such as searching for specific items, or making repairs. Read the input from a file, and use as many input files as needed.

Input file examples

items.txt

- Format

ID Title Author Page_Count Topic Type Copies

- Sample input

```
1 Clean_Code Robert_C_Martin 464 Software_Engineering Book 3
2 The_Pragmatic_Programmer Andrew_Hunt 352 Software_Engineering Book 2
3 Design_Patterns Erich_Gamma 395 Software_Engineering Book 2
4 Refactoring Martin_Fowler 448 Software_Engineering Book 3
```

members.txt

- Format

ID Name Email Category

- Sample input

```
1 Bence_Nagy bence@elte.hu Student
2 Eszter_Kovacs eszter@elte.hu Faculty
3 John_Smith john@elte.hu Alumni
4 Levente_Toht levente@elte.hu Student
5 Sophia_Martinez sophia@elte.hu Faculty
```

2 Recipe Book System for a Restaurant

The Recipe Book System is designed to provide restaurants with a structured and extensible way to manage their recipes and their inventory to deliver the best and most fresh food.

The Recipe book holds a list of all available recipes. New recipes can be added and removed. Also, people can search for recipes in the system using different criteria, such as search by a specific ingredient. The system provides users with the ability to search and filter recipes by name, cuisine, ingredient, offering a highly customizable browsing experience.

Each recipe has its name, cuisine, list of ingredients, and cooking time. For each recipe you can add and remove ingredients, calculate total cooking time, and get recipe details. We are not modeling quantities of the ingredients, each recipe requires exactly one of each.

Each ingredient has a name, category, and unit of measurement. The ingredients are classified into categories such as dairy, meat, or vegetables, making it easier for chefs to organize and filter their recipes. By grouping ingredients according to their category, chefs can be more efficient. Also, you need to keep track of how many recipes an ingredient is used.

The restaurant keeps track of its recipes through a recipe book, its ingredients in stock and a shopping list. To prepare a recipe, the restaurant must check if all ingredients are available in stock; if not available then the restaurant cannot prepare the recipe and the ingredient is added to the shopping list, so that it would be bought later. If all ingredients are available, then the recipe is prepared, and the ingredients are taken from the stock. The recipe book contains a list of all recipes of the restaurant. And you can search for the most used ingredients, and the most popular categories of ingredients in this recipe book

In the ingredient Stock, for each ingredient you need to know the stock of each ingredient, add stock whenever needed, check if an ingredient is available or not. Make sure to be able to reduce stock whenever a recipe is being prepared.

The shopping list contains the list of ingredients that must be bought. New ingredients are added when there is no more in stock. When buying new ingredients either the whole shopping list is bought, or sometimes only an item can be bought. When item is bought it should be removed from shopping list and added to stock again.

Simulate the process by initializing a restaurant object and adding a recipe book to it. The restaurant also has a shopping and stock lists. Chefs should be able to cook new recipes by using ingredients available in stock,

and when needed items on shopping list should be bought. Read the input from a file, and use as many input files as needed.

Input file examples

ingredients.txt

- Format

Name Category Unit_Of_Measurement

- Sample input

Spaghetti Pasta Grams
Ground_Beef Meat Kilograms
Tomatoes Vegetables Kilograms

recipes.txt

- Format

Name Cuisine Ingredients Cooking_Time

- Sample input

Spaghetti_Bolognese Italian Spaghetti-Ground_Beef-Tomatoes-Garlic-Olive_Oil 30
Chicken_Alfredo Italian Chicken-Alfredo_Sauce-Garlic-Parmesan-Fettuccine 25
Veggie_Burger American Veggie_Patty-Lettuce-Tomato-Onion-Bun 20

3 Venue Booking System in University

The university provides different types of venues that can be booked by anyone with a university email, including staff and students. The booking of all venues is done through one system per university. Each venue belongs to a specific department of the university. To ensure serving different purposes and various events, some venues have specific equipment installed, and depending on each department, other equipment could be requested.

Venues are managed by departments (for example: Informatics, Languages, etc.). A department has a physical location. Each department also has a contact person, if help is needed when making reservations this person's information would be available. The department has a list of available equipment that could be rented when a venue reservation is made. A department can have multiple venues or no venues, but a venue must belong to a department.

For each venue, the venue should have a name, capacity, and a count of bookings is also kept. Each venue may be managed by a department. A routine maintenance check is performed after every 5 bookings. If a venue requires additional repairs beyond standard maintenance, a maintenance request is generated. Users should be able to get details of the venue. Double booking should not be allowed.

Each piece of equipment has a name (projector, microphone, etc.), and a type (audio, visual, furniture), a brand, and extra price (can be zero). Some venues come with permanently installed equipment that cannot be rented separately (you don't need to model these). However, departments offer additional rentable equipment that users can request when booking a venue. Departments track rental usage, including frequently requested items and revenue collected.

Users must be university members verified by their email. The user type is one of these: admin, student, or faculty. Users can make, cancel, and view reservations, with a total count of their bookings tracked.

The university has a single centralized booking system that manages all venue reservations. The Booking System has a list of all available venues at the university. It has the following functions:

- Browse available venues and check real-time availability.
- View venue details, including installed equipment and rental options.
- Make, modify, or cancel bookings based on their user role. Request additional equipment from the venue's managing department.

- Track maintenance schedules and report venue issues.

The main simulation must initialize the University and its departments. For each department add the venues and the equipments. Add users so that they can make booking of venues and request extra equipment using the booking the system. Read the input from a file, and use as many input files as needed.

Input file examples

users.txt

- Format
Name Email Type
- Sample input

```
Sandor sandor@elte.hu admin
Peter peter@elte.hu admin
Anna anna@bme.hu faculty
Zoltan zoltan@corvinus.hu student
David david@elte.hu faculty
Eva eva@bme.hu student
```

departments.txt

- Format
Name Location ContactPerson
- Sample input

```
IK Lágymányos Sandor
TTK Lágymányos Peter
```

venues.txt

- Format
Name Capacity Department Equipment
- Sample input

Main_Lecture_Hall 200 IK Projector-Microphone-Whiteboard
 Small_Meeting_Room 20 IK TV_Screen-Conference_Phone
 Physics_Lab 50 TTK Lab_Benches-Fume_Hood
 Business_Conference_Room 40 IK Smart_Board-Video_Conference_System
 Workshop_Hall 80 TTK Work_Benches-3D_Printer

equipment.txt

- Format

Name Type Brand RentalPrice Department

- Sample input

Projector Visual Epson 5000 IK
 Microphone Audio Shure 2000 IK
 Whiteboard Furniture Nobo 0 IK
 Conference_Phone Audio Polycom 3000 IK
 TV_Screen Visual LG 4000 TTK
 Smart_Board Visual Samsung 7000 TTK
 Fume_Hood Furniture Thermo_Fisher 0 TTK
 3D_Printer Visual Prusa 15000 TTK

4 Orion Expedition

In the vast reaches of space, a powerful space shuttle embarks on a voyage to explore distant planets. To navigate this journey, the shuttle relies on several key systems that work together to reach its destinations efficiently.

Destinations are planets or celestial bodies the shuttle aims to reach. Each destination has a name, a distance from Earth, and a travel progress to track the shuttle's travel progress towards it. The destination also has a formula to calculate how far the shuttle is from the destination, so that it can update its progress. Here we assume that all planets are in a line, so the distance is just "distance of destination from Earth" - "how far we have come from Earth".

The navigation system acts as the shuttle's guide, maintaining a list of destinations. It allows adding new destinations and removing ones that have been fully reached. The navigation determines which destination is nearest to the shuttle, helping it plan the most efficient route.

The tank is responsible for fuel management: it keeps track of the current fuel and the maximum fuel capacity. The tank reduces the available fuel based on a percent usage, while refueling restores the tank to full capacity. If the fuel drops below 15% of its maximum level, the system signals that it is low on fuel. Of course, the tank can check how much fuel is left.

The main engine provides the thrust needed for space travel. It can start and stop as required, but moving reduces fuel by 10% and increases the shuttle's progress toward its next destination by 5%.

The orbiter is the shuttle's navigator. It has current destination that starts with planet Earth, a next destination, and a reference to the shuttle. To start the course, the orbiter takes the next destination, starts the engine, and initiates movement while the engine is running. If fuel runs low, it attempts to refuel at 50% chance before continuing. Otherwise, the engine stops running. If a shuttle reaches its target by then, the orbiter updates its current destination.

At the heart of it all is the space shuttle, which ties these components together. It has a navigation system, an orbiter, an engine, and a fuel tank. The shuttle can add destinations and launch into space, driving the orbiter towards the closest destination. When a destination is successfully reached, it is removed from the navigation. The launch is complete when all destinations have been reached.

The main simulation serves as the entry point, initializing the space shuttle, setting destinations like Mercury, Jupiter, Saturn, Neptune, and Mars, and launching the expedition. The shuttle systematically travels from one

destination to another, overcoming fuel challenges along the way. Once all planets are visited, the mission is deemed a success. Simulate, launch the shuttle, and show the actions!

Input file examples

destinations.txt

- Format
Name Distance
- Sample input

```
Jupiter 1000  
Saturn 500  
Neptune 200  
Mars 4000
```

5 Hospital Emergency

In a bustling city, emergencies strike at unexpected moments. When a medical crisis occurs, the city's emergency alert system ensures that patients receive the care they need as quickly as possible. The system relies on ambulances, hospitals, and doctors working together to save lives.

Patients are individuals in need of medical attention. Each patient has a name, a severity level ranging from 1 (mild) to 10 (critical), and a status indicating whether they have fully recovered. When a doctor treats a patient, their severity level is reduced based on the doctor's efficiency. If the severity reaches zero, the patient is marked as treated. This process not only reduces the severity level but also checks if the patient has recovered.

Ambulances transport the patients. Ambulances can be distinguished between each other; each has a status indicating if it can transport a patient, and a destination hospital where it will take the patients.

Hospitals are the core of the system, where treatment takes place. The Hospital represents a medical center with a name, a capacity defining the number of patients it can accommodate, and a list of both patients and doctors. The hospital registers a patient if there is space and can also register a doctor to the hospital's staff. When it is time for treatment, the hospital instructs all doctors to care for all untreated patients.

Doctors are responsible for treating patients. A doctor has a name and an efficiency level, which determines how much the doctor can reduce a patient's severity per session.

Overseeing all these operations is the emergency alert system. It manages all of the hospitals and ambulances, ensuring that patients are taken to available hospitals. The system allows new hospitals and ambulances to join the system, and retrieves a hospital not at full capacity, if any. When a patient needs transport, the system finds an available hospital and the first ambulance available to transport them. If no hospitals have space or no ambulances are free, the patient must wait. Once patients arrive at hospitals, the system ensures that doctors begin their work.

The main simulation ties everything together. It initializes the alert system, registers hospitals and doctors, assigns ambulances, and simulates patient arrivals. Simulate, start the treatments, and show the actions!

Input file examples

`hospitals.txt`

- Format

Name Capacity

- Sample input

General_Hospital 3
City_Medical_Center 2

doctors.txt

- Format

Name Experience

- Sample input

Dr_Smith 5
Dr_Adam 3
Dr_Farizi 8

ambulances.txt

- Format

ID

- Sample input

A1
A2
A3
A4

patients.txt

- Format

Name Severity_Level

- Sample input

John 7
Alice 4
Bob 9
Eve 6
Charlie 3

6 Adventurer's Guild

In a world of adventure, a guild recruits adventurers to take on dangerous quests. Each adventurer arms themselves with weapons and faces formidable monsters to prove their strength.

Weapons represent equipment that boost an adventurer's power. A weapon has a power boost, which increases attack strength, and a durability, which determines how many times the weapon can be used. The weapon can decrease its durability, and checks if it is unusable. The weapon can also return the power increase, if it is still functional.

Monsters are the creatures that adventurers battle. They can be identified by a name, have strength, health, and armor. The monster reduces its health based on incoming damage and its armor, and can check if its health reaches zero (then it dies). The monster can also return its attack power and defense.

An adventurer is a hero with a name, base power, experience, rank, health, and an inventory of weapons. They can equip weapons which boost their attack power. The damage of the adventurer's attack is calculated from its total strength—base power * (10% of rank) + total strength of weapons. The adventurer's health can be decreased, and they can fall in battle if it reaches zero. After each battle, the adventurer fully restores their health. The adventurer use their weapons in combat, gain experience, and increment their rank and power up by 20% when enough experience is gained—which is when their experience is more than or equal to tenth of their rank. The adventurer can also initiate a quest for themselves.

A quest represents a mission with a name, difficulty, a status to check if it's completed, and a list of monsters. The quest is done by an adventurer, who fights each monster sequentially. If an adventurer succeeds, the quest is marked complete. Otherwise, it is incomplete. The adventurer takes turns to combat the monster, alternating attacks until one is defeated. When the adventurer attacks, it uses its equipment first (so it loses durability), then attacks (damage = power - monster's armor), and if the monster is not defeated, they receive damage from the monster. A defeated monster will let adventurer gain experience.

Guilds manage adventurers and quests. It has a list of adventurers and a quest board. The guild can add a new member and introduce new quests to the adventurers. The guild assigns a random adventurer to quests, letting them attempt challenges sequentially until all quests are completed.

The main game runs the simulation. It establishes a guild, recruits the adventurers, equips them with weapons, and posts quests. Simulate, start

the quests, and show the actions!

Input file examples

adventurers.txt

- Format
Name Health Weapon_Attack
- Sample input

```
Aragorn 50 30  
Legolas 40 20  
Gimli 60 40
```

quests.txt

- Format
Name Difficulty Monsters
- Sample input

```
Defeat_the_Goblin_King 70 Goblin_Warrior-Goblin_Warrior-Ogre-Goblin_King  
Rescue_the_Village 50 Ogre-Ogre-Bandit_Leader  
Slay_the_Dragon 100 Fire_Dragon
```

monsters.txt

- Format
Name Health Attack
- Sample input

```
Goblin_King 70 100  
Goblin_Warrior 40 50  
Ogre 50 80  
Bandit_Leader 60 70  
Fire_Dragon 1000 200
```

7 Borderland Survival

In a distorted other world known as the Borderland, some creatures are forced to play deadly games to survive. Every game tends to test intellect, brawn, teamwork, or brutality. Players must scavenge for supplies, form alliances, and solve increasingly complicated puzzles in their efforts to remain alive (forming alliances and scavenging are not modeled).

Every player has a name, a mental stat, a physical stat, and a fear level. Players may gain or lose stats based on game events. When the fear level reaches a certain threshold, the player will freeze and become inactive.

Each game has a name, a type (mental or physical), and a difficulty level. Mental games require the mental stat, and physical ones the physical stat. The game can be played by a group of players and everyone whose appropriate stat is above the difficulty level wins. Players who win will be less afraid, and players who lose will be more afraid.

Players are rewarded by cards for winning a game. All cards possess a suit and number (i.e., 7 of Spades, 3 of Hearts), and having all 52 cards will enable one to exit.

Games are occurring in an arena. The arena has a name, a series of games, and an environmental hazard which affects players (e.g., fire, building collapse, mist). When the hazard activates, players become more afraid.

The Game Master controls the games. They control the hazards in arenas and pick the next game randomly to create suspense.

The larger flow is controlled by the Borderland Manager. It sets up everything for the games, advances time by starting a game, and checks if the games are over. It can also observe the players and print them for the viewers.

The game ends when all the cards have been collected or when all the players are inactive. Play your survival, test yourself to limits, and try to get out of the Borderland!

Input file examples

players.txt

- Format
Name Mental Physical
- Sample input

Maksud 7 5
Farizi 6 8
Silva 9 3

games.txt

- Format
Name Type Difficulty
- Sample input

Bridge_Puzzle mental 6
Tag_Arena physical 5
Quiz_of_Death mental 7

arena.txt

- Format
Name Hazard
- Sample input

Abandoned_Theme_Park Fog

8 Wildlife Simulation

Wildlife conservation is the most critical issue in a major game reserve. Trained rangers venture out daily to spot wildlife, administer medicine to diseased animals, and protect the habitat from desecrating poachers. The reserve is a well-organized company where rangers, animals, habitats, and support vehicles coexist in symbiosis for fostering nature's fragile balance.

All animals have a species, health, and a stress level. Animals become stressed or injured due to natural events or poachers. The rangers can calm them down and treat them.

Rangers are diligent workers who protect the reserve's environment. Every ranger has a name, experience, and efficiency. They can assist an animal by improving its status or reducing stress. They can also detect animals that need help among all the animals and help them. If they are in the same habitat as a poacher, they find and fight the poacher. The winner is determined based on the strength of both the ranger and the poacher. The strength is initialized randomly.

Poachers are intruders who hunt down animals. Each poacher targets a specific species. Different poachers are dangerous to a different degree. Poachers disturb wildlife and cause them stress as they evade their hunters. They can also hurt the animals if they catch them.

Each habitat has a name, a capacity, and the animals living in it. Animals can enter and leave habitats. If there is an incursion by a poacher, it stresses all animals in the habitat.

The rangers travel in Jeep vehicles. The vehicle has an id, fuel level, and a capacity. Vehicles can be refueled and deployed to specific habitats.

The simulation is run day-by-day. Each day, animals wander between habitats, rangers are sent to habitats to patrol them, and poachers go to habitats to hunt animals. The simulation reports at the end of the day the state of the reserve, and the happenings on that day, for example, what animals were saved. Play it out, roll out your rangers, and save the wildlife!

Input file examples

animals.txt

- Format
Species Health StressLevel
- Sample input

Elephant 60 30
Giraffe 70 20
Zebra 80 25

rangers.txt

- Format
Name Experience Efficiency
- Sample input

Balazs 5 7
Sofia 6 6

vehicles.txt

- Format
ID FuelLevel Capacity
- Sample input

V1 100 2
V2 100 3

habitats.txt

- Format
Name Capacity
- Sample input

Savannah 3

9 Racing Tournament Simulation

During a high-speed race competition, professional drivers compete with diverse cars on challenging tracks. Every race continues to set new records of speed, agility, and endurance. Backstage, a system oversees cars, drivers, tracks, and the entire competition flow — from preparation to results.

Each vehicle has a model, speed, durability, and fuel. When a vehicle is driven, it consumes some amount of fuel and its durability is reduced. If durability or fuel drops too low, the vehicle becomes less effective. It can also become non-functional in case of very low durability.

Every driver has a name, skill, and a preferred vehicle model. The effective speed for a driver driving the vehicle is calculated based on driving skill and the maximum possible speed of the vehicle model. If a driver drives their preferred model their effective speed increases by 10%. Drivers pit stop to refuel or repair if needed in the course of a race.

A racetrack has a name, difficulty level, and a number of laps. A given driver and car can compete on the track, lap racing and keeping track of gasoline, durability, and speed. The difficulty level influences fuel consumption and wear and tear of the vehicle.

Mechanics offer the support needed to race. Each mechanic has a name and efficiency. Cars can be repaired or refueled by mechanics. Mechanics are provided to offer assistance to individual drivers on a race-by-race basis.

Race events are planned races with drivers attending and a track. After a race, the results are reported, and points are distributed to the drivers according to their placements.

The race events are organized into a tournament, where all of the drivers participate in each race. Each track is raced on exactly once in the tournament. The results of the tournament are reported after each race, and also at the end. The 1st, 2nd, and 3rd drivers are applauded at the end on the podium.

Race to victory!

Input file examples

drivers.txt

- Format
Name Skill PreferredVehicle
- Sample input

Maksud 8 Ferrari
Farizi 6 BMW
Silva 7 BMW

vehicles.txt

- Format
Model Speed Durability Fuel
- Sample input

Ferrari 100 90 100
BMW 85 80 100
Porsche 95 85 100

mechanics.txt

- Format
Name Efficiency
- Sample input

Mike 5
Anna 7
Luca 6

tracks.txt

- Format
Name Difficulty Laps
- Sample input

Desert_Drift 10 3
Urban_Dash 12 4